When LLMs Generate Correct Security Patches for the Wrong Reasons: Benchmark and Detection

Lam Nguyen Tung, Aldeida Aleti, Xiaoning Du, Neelofar Neelofar, Matias Martinez, Justyna Petke, Federica Sarro

***Abstract*—LLMs have shown remarkable progress in automated vulnerability repair (AVR). Current AVR systems primarily focus on generating patches that satisfy conventional correctness criteria, such as passing test suites, preserving functionality, and eliminating vulnerabilities. However, these criteria do not necessarily reflect LLMs' true understanding. Research has shown that LLMs can rely on spurious correlations in training data or on memorization. This allows them to generate correct patches without truly understanding the semantic root cause of vulnerabilities. Existing AVR evaluations may therefore overestimate model capability, as repair success may reflect superficial pattern recall rather than genuine reasoning. This limitation leads to *untrustworthy* predictions. Consequently, when deployed in the real world with unseen vulnerabilities, the performance of AVR systems often deteriorates significantly. Hence, evaluating and detecting untrustworthy predictions is crucial. This, however, remains underexplored.**

This paper investigates this gap by introducing the first benchmark and automated detection approach for untrustworthy predictions in LLM-based AVR. We construct VFXplainBench, a benchmark of 11,210 predictions generated by multiple systems, both commercial and open-source LLM backends. We find that 17.2% of correct patch predictions are actually untrustworthy. To detect them, we propose SUSVF, an automated method that identifies inconsistencies between the generated patch and its explanation, within the explanation's reasoning steps, and between the explanation and the vulnerable code. Evaluated on VFXplainBench, SUSVF achieves 0.882 precision, 0.732 recall, and 0.768 MCC, substantially outperforming LLM-as-a-Judge. These results show that patch correctness alone is insufficient for evaluating AVR systems. They also highlight the importance of explanation-based untrustworthiness detection for safer deployment of LLMs.

***Index Terms*—Vulnerability Repair, Explanation, Trustworthy**

I. INTRODUCTION

Large language models (LLMs) have revolutionized automated vulnerability repair (AVR) [1], achieving state-of-the-art (SOTA) success rate [2]. LLM-based AVR systems mainly focus on conventional correctness criteria, such as passing tests, preserving functionality, and eliminating vulnerabilities. However, these criteria alone are inadequate, as they do not necessarily reflect LLMs' true understanding and overlook issues in their decision-making. Studies have shown that LLMs can be untruthful [3], misunderstand vulnerabilities [4], hallucinate [2], memorize fixes [5], or rely on spurious correlations [3]. As a result, they may generate correct patches for the wrong reasons. We call such cases *untrustworthy* predictions[1].

The Vulnerable Code

```
decrypt_response(unsigned char *out, size_t* out_len) {...
  while (0x80 != plaintxt[cipher_len - 2]
        && cipher_len - 2 > 0)
    cipher_len--;
  if (2 == cipher_len)
    return -1;
  memcpy(out, plaintxt, cipher_len-2);   Vulnerable
  *out_len = cipher_len - 2;
  return 0;
}
```

Vulnerability Repair Prediction

Vulnerability Analysis: The buffer "plaintext" is copied to "out" with size "cipher_len - 2". However, if "cipher_len" is less than 2, "plaintext" will be out of bounds.

Logically consistent

Fixing Action: We should ensure that "cipher_len" is greater than 2, which can be done by adding a check before the copy operation.

Inconsistent between Explanation–Patch

Therefore, the patch should be:

```
- if (2 == cipher_len)
+ if (2 == cipher_len || *out_len < cipher_len-2)
```

Fig. 1: A program with the CVE-2018-16421 vulnerability (top), where writes may occur outside the buffer `out`'s intended boundary, denoted by `*out_len`. The AVR prediction made by DeepSeek-Coder-33B-Instruct (bottom) provides a patch and its explanation. Although the patch is correct and the explanation itself is logically consistent, the explanation proposes a fixing action inconsistent with the patch.

It is important to detect untrustworthy predictions by assessing LLMs' decision-making process [6]. If left undetected, untrustworthy predictions can inflate LLMs' perceived capability during evaluation. When deployed on unseen vulnerabilities, the performance of LLMs often deteriorates significantly [6]. Detecting untrustworthy predictions helps practitioners identify behavioral issues, better grasp the true AVR capability and avoid overtrust that may lead to unreliable patches.

LLM explanations, e.g., chain-of-thought (CoT) intermediate reasoning steps [7], have emerged as an important tool to facilitate the detection of untrustworthiness in AVR [6], [8], [9]. They explicitly illustrate the reasoning process by which LLMs generate patches, expressed in natural language (NL). By exposing how a patch is derived, explanations allow us to audit, diagnose, and filter untrustworthy predictions. Incorrect explanations can indicate untrustworthy predictions.

Example. Figure 1 shows a C program affected by the CVE-2018-16421[2] vulnerability. The program tries to copy `cipher_len-2` bytes from `plaintxt` to the buffer `out`.

[1]We define a prediction an LLM's generation of a patch and its explanation

[2]https://nvd.nist.gov/vuln/detail/CVE-2018-16421

The vulnerability occurs if it writes to a memory location outside `out`'s intended boundary, denoted by `*out_len`. The figure also includes a prediction made by DeepSeek-Coder-33B-Instruct, which contains a patch and its explanation. The patch is correct, adding a boundary check to avoid `*out_len < cipher_len-2`. However, while the explanation itself seems logical, it proposes a fixing action different from the patch. The inconsistency between the patch and explanation makes this prediction untrustworthy, likely because the model memorised the correct patch during training.

In hindsight, the LLM seems to work properly. However, the explanation reveals the LLM fails to capture the root causes of the vulnerability and only addresses it in a spurious way [5]. This could leave vulnerabilities with similar root causes unpatched. Moreover, security auditors can be misled by the explanation, leading to programming skill degradation [9], They can make poor decisions about future code changes or propagate flawed reasoning to related components.

Our Goal. LLM-provided explanations can act as a useful tool for filtering out untrustworthy AVR predictions. However, reviewing them remains manual, making it time-consuming and unscalable. This motivates the need for a mechanism that can flag untrustworthy predictions automatically.

Benchmark. To evaluate and characterize the untrustworthiness of LLM-based AVR, we construct VFXplainBench, a benchmark of multiple SOTA systems: instruction prompting [2], [10], APPATCH [11], [12], and Vul-R2 [13], using various LLMs backends. We collect 11,210 AVR predictions made by them on their evaluated datasets, including ExtractFix [14], ZeroDay [11], SVEN [15], and PrimeVul [16]. By manually comparing with developer-created patches, we identify 1,678 correct patch predictions. Among them, we identify trustworthy predictions by manually validating the explanations against the developer-created patches, vulnerability reports, and messages from the fixing commits. VFXplainBench shows that 11.4%–24.9% of correct patch predictions are untrustworthy, even with the most advanced LLM backends, such as GPT-4 and Claude-Opus 4.1. Further analysis reveals various recurring issues that can be systematically categorised into ① the explanation inconsistent with the final patch, ② conflicts between the proposed fixing action and the vulnerability analysis in the explanation, and ③ hallucinated identifiers that are used in the patched code but without introduction in the explanation. These issues provide a checklist to detect untrustworthy predictions.

Key Insight. We ultimately aim to ease the burden of manual review by automatically identifying whether a correct patch is generated in a trustworthy manner. Our key insight is that the explanation of a trustworthy prediction should be not only logically consistent (i.e., the proposed action can fix the analysed vulnerability) but also consistent with both the patch and the vulnerable code. Such consistencies alone may not guarantee correctness and trustworthiness. However, inconsistencies serve as a strong *sufficient condition* for untrustworthiness, which enables the detection of untrustworthy predictions. We believe it is theoretically infeasible to exhaustively identify all untrustworthy predictions, as it would require intelligence and knowledge similar to or exceeding that of AVR itself. With inconsistencies, we can leverage them to provide concrete evidence of untrustworthiness. Predictions that exhibit these inconsistencies are definitely untrustworthy. Accordingly, *if an explanation is not consistent, its corresponding patch prediction is untrustworthy.* In this way, we can precisely identify untrustworthy predictions with minimal false alarms.

Approach. We develop SUSVF, the first automated approach that can **S**pot **u**ntru**s**tworthy LLM-based predictions for **V**ulnerability **F**ixes. SUSVF detects untrustworthy predictions through a three-stage process that assesses inconsistencies across ① the patch and the explanation, ② the explanation's reasoning steps, and ③ the explanation and the vulnerable code. While ③ can be assessed by rule-based and static code analyses, ① and ② can be evaluated by learning-based semantic analysis and natural language inference (NLI). Learning-based approaches may themselves raise trustworthiness concerns. Hence, we deliberately reduce their scope to tasks substantially simpler than AVR, such as semantic similarity measurement, short-document summarisation, and basic NLI, which can be reliably handled by current SOTAs. With that in mind, we carefully design criteria and apply multiple techniques to enhance SUSVF's own trustworthiness.

In Stage ①, SUSVF generates an NL description of the code changes in the patch by summarising fine-grained differences identified by tools (e.g., Gumtree [17]). The explanation–patch inconsistency is estimated by semantic equivalence between the generated description and the explanation. To enhance the accuracy of this estimation, we use majority voting over multiple metrics, e.g., rouge [18] and cosine similarity. In Stage ②, SUSVF reframes the evaluation of the logical inconsistency between an explanation's reasoning steps as an NLI task. Given a vulnerability trigger, i.e., the condition under which it occurs, and a proposed fixing action explicitly described within the explanation, the task is to assess whether the action can eliminate the trigger. Unlike AVR, which requires extensive domain knowledge, we leverage LLMs' NLI capabilities by rigorously prompting them with the trigger and action to assess their logical inconsistency. In Stage ③, we apply static code analysis to compare identifiers in the vulnerable and the explanation, with the fixed code as a supplementary reference. SUSVF identifies if there exist unknown identifiers not introduced in the explanation. Finally, a prediction is labeled untrustworthy if inconsistency is observed in any stage.

We evaluate SUSVF on VFXplainBench. Results show that SUSVF achieves good effectiveness of 0.882 in Precision, 0.732 in Recall, and 0.768 in Matthews correlation coefficient (MCC). It also outperforms LLM-as-a-Judge [19] methods by up to 589% in Precision, 4,256% in Recall, and 4,367% in MCC. We summarise our contributions as follows:

- A trustworthiness benchmark, VFXplainBench, of 11,210 predictions made by multiple LLM-based AVR systems;
- An approach for automated detection of untrustworthy AVR predictions, SUSVF, which in our experiments achieved 0.882 precision and 0.732 recall;
- A comprehensive evaluation of SUSVF, showing its superior effectiveness in detecting untrustworthy AVR predictions compared to other approaches; and
- A replication package: **zenodo.org/records/18403393**.

II. Preliminary and Background

A. Definitions

1) Explanation: As LLMs are inherently black-box, several techniques have been explored to provide explanations for their predictions. Recently, Chain-of-Thought (CoT) [7] and its variants [20], [21], have gained popularity. They explicitly outline sequences of intermediate reasoning steps that lead to a final answer. CoTs help LLMs tackle complex tasks by breaking them into multiple steps, offering interpretable windows into model behavior [7]. For these reasons, CoT prompting has been widely applied to various SE tasks, including AVR, to enhance performance and provide explanations [2].

In this study, we refer to an *explanation* as CoT intermediate reasoning steps expressed in NL. They often describe vulnerability semantics based on relevant lines of code, analyse the root cause of the vulnerability, and propose a fixing action. An illustrative explanation is shown at the bottom of Figure 1.

Keep in mind that these explanations differ fundamentally from post-patch explanations. A post-patch explanation is generated independently and serves as a retrospective justification for an already-determined patch, rather than reflecting the LLM's reasoning. As a result, unlike CoT intermediate steps that arise during generation, post-patch explanations may not faithfully represent the reasoning that generated the patch. This makes the evaluation of LLM untrustworthiness less reliable.

2) Trustworthiness: In the context of LLMs, trustworthiness is often referred to as their intrinsic quality, which can be defined as a combination of robustness, security, transparency, fairness, and safety [9], [22]. This paper focuses on trustworthiness, in particular, whether an LLM predicts correct patches for the right reasons rather than spurious ones. Hence, we adopt the definition of trustworthiness proposed by Kästner et al. [23] as shown in Definition 1.

Definition 1. A system is trustworthy to a stakeholder in a given context iff (a) the system works properly in the context and (b) the stakeholder would be justified to believe (a).

According to Definition 1, understanding a system thoroughly helps justify our beliefs about how well it works. Explanations can serve as a means to gain this understanding, thereby promoting trustworthiness [23]. In the context of AVR, a trustworthy AVR system should predict correct patches, and explanations behind their predictions should also be correct. We propose the definition of a trustworthy AVR prediction.

Definition 2. A trustworthy AVR prediction generates a correct patch and the explanation behind it is also correct.

A prediction may fall into one of three cases: (1) an incorrect patch, (2) a correct patch with a correct explanation, or (3) a correct patch but with a wrong explanation. In (1), investigating patch incorrectness becomes more important than trustworthiness testing. Patch correctness assessment has been widely studied [24], and is not the focus of this paper. Following prior work [6] in trustworthiness testing, we instead differentiate the last two cases. This means that we examine only correct patch predictions: (2) represents trustworthy predictions, while (3) indicates untrustworthy ones.

3) Inconsistency: Adapted from prior work on LLM hallucination [25], a correct explanation for a correct patch should remain logically coherent and well-aligned with both the patch and code context. We believe it is theoretically infeasible to exhaustively verify this, as it would require intelligence and knowledge similar to that of AVR itself. For that reason, we treat inconsistency as a *sufficient condition* for incorrect explanations, which is later leveraged in Section III–IV.

There are three main elements in the AVR process, including the vulnerable code, the patch, and the explanation. Hence, three types of inconsistencies can arise: inconsistency between the patch and the explanation, inconsistency in the explanation's reasoning steps, and inconsistency between the vulnerable code and the explanation. We do not consider the relationship between the vulnerable code and the patch in this paper, as we only focus on correct patch predictions and assume they are consistent.

Definition 3. An explanation is inconsistent if it exhibits any violation of alignment with respect to the predicted patch, its own reasoning steps, or the vulnerable code.

In this paper, inconsistencies are a form of non-uniformity. We will describe how to quantify them in Section IV. Note that inconsistency serves as a sufficient (but not necessary) condition for untrustworthiness. That is, if an explanation is inconsistent, the corresponding prediction is definitively untrustworthy. This enables high-precision detection of untrustworthy predictions, albeit potentially at the cost of recall. Notably, untrustworthy predictions may produce consistent explanations in some specific cases. For example, Appendix C shows an untrustworthy prediction: its explanation is consistent, but the described root cause of the vulnerability is actually inadequate.

B. Application of Untrustworthiness Detection

The phenomenon of untrustworthiness, where LLMs generate correct answers for the wrong reasons, exists in various domains [5], [6], including AVR [2]. LLMs often achieve a good repair success rate on observed AVR benchmarks. However, when deployed in the real world, they are unlikely to perform well, particularly on vulnerable code that falls beyond their training cutoff. This discrepancy suggests that benchmark performance may inflate the actual capabilities of the LLMs. Hence, it is also crucial to assess the reasoning of LLMs to detect untrustworthiness prior to deployment [6].

Prior studies [26], [27] indicate a strong preference among developers to remain involved in the repair process, rather than relying on fully automated suggestions. This preference is mainly due to trust issues with automatically generated suggestions. In particular, untrustworthy predictions may be disregarded, as developers need to correctly understand the rationale behind the recommendations to trust and implement them effectively [2]. Traditionally, developers often detect untrustworthy predictions by manually reviewing LLMs' explanations accompanying the patches, which is not scalable [6].

Automating this process in the software engineering (SE) lifecycle of LLM-based AVR systems can greatly advance their development and applications. Automated detection

methods facilitate the review process of AVR predictions. They also enable the automation of trustworthiness testing, which can be integrated into continuous integration and delivery pipelines. These activities are vital and closely intertwined with others in SE [28]. This is especially valuable in the iterative development of LLM-based systems where performance and trustworthiness must be evaluated and refined [29].

Beyond development, the detection methods also serve as a tool for monitoring LLM-based AVR systems in real-world environments, particularly in online testing or DevOps workflows. They can evaluate whether predictions are untrustworthy in real time, fostering greater human trust. They can also test real-world and corner-cased inputs. Moreover, the outputs of such detection methods provide actionable insights for feedback-driven improving LLM-based AVR systems.

III. VFXplainBench: Trustworthiness Benchmark of LLM-based Vulnerability Fixes

As far as we know, no benchmark exists for the untrustworthiness of LLM-based AVR. Hence, to evaluate and characterize its untrustworthiness, we first construct VFXplainBench, a trustworthiness benchmark of multiple SOTA LLM-based AVR systems. We collect their predictions and manually identify untrustworthy ones. By doing so, we aim to investigate the first research question (RQ) as follows.

RQ1 [Benchmark]. *Are vulnerability fixes generated in a trustworthy manner?* Section V-A later introduces **RQ2–RQ5** for detecting untrustworthiness using VFXplainBench.

A. Model Selection

We select LLM-based AVR tools peer-reviewed and published within five years at top venues in SE, AI, and Security. We include 13 conferences (ICSE, FSE, ASE, ISSTA, CCS, S&P, USENIX Security, NDSS, AAAI, IJCAI, ICML, NIPS, ICLR) and three journals (TOSEM, TSE, EMSE) [2]. We then conducted a manual search to find relevant work using keywords *("LLM" OR "Large Language Model") AND ("Vulner*" OR "Secur*") AND ("Repair" OR "Fix" OR "Patch")* [2]. We focus specifically on tools that take vulnerable code as input and generate both a security patch and an NL explanation as output. At the time of writing, only a limited number of such tools are available, as most existing ones do not provide explanations. In particular, we exclude VulRepair [30], VulMaster [1], PatchAgent [31] and their variants, as they only generate patches. We also exclude San2Patch [32]. Although it provides explanations, it requires sanitizer logs as input. Consequently, we choose the following LLM-based AVR tools in our study.

- **Instruction Prompting** (IP) [2], [10] enables off-the-shelf LLMs to address complex tasks like AVR. We follow prior studies to use CoT prompting that emulates the multi-step vulnerability analysis process of human security experts. Prior studies [33] observed that the experts often: first gain an overview of the code, analyse critical sub-components to pinpoint the vulnerability trigger, i.e., the condition under which it occurs, propose a patching action to eliminate the vulnerability, and finally create a patch [4]. Accordingly, we instruct the model to follow these four steps. This allows LLMs not only to generate patches but also to provide explanations that justify their predictions. Each prompt includes the vulnerable function, the common weakness enumeration ids (CWE-IDs), and the vulnerability location [12].
- **AppPatch** [11] is an approach for vulnerability semantic reasoning and adaptive prompting on LLMs. It first applies semantics-aware scoping to isolate a minimal yet sufficient slice of code for analysis. Within this reduced scope, AppPatch uses vulnerability semantic reasoning to identify the likely root cause and dynamically retrieves relevant exemplars from a pre-mined database of human-written patches to guide the model's reasoning. It then automatically applies these contexts and few-shot CoT prompting to produce vulnerability patches and explanations.
- **Vul-R2** [13] is a fine-tuned reasoning LLM for vulnerability repair. It consists of two key components: a domain-aware reasoning learning module and a curriculum-based verifiable rewarded training module. The former comprises a reasoning answer construction component, a reasoning data filtering process, and a supervised fine-tuning process for learning vulnerability-related reasoning knowledge. The latter comprises dynamic reinforcement learning with verifiable rewards paradigms based on multiple-choice question answering and character-level matching. Vul-R2 can generate patches and step-by-step reasoning explanations.

TABLE I: Except for Vul-R2, we use the following LLMs as backends for Instruction Prompting and AppPatch

Model		Params	Type
GPT-4	gpt-4-0613	1.76T	Closed-source
Claude-Opus 4.1	claude-opus-4-1-20250805	-	
Qwen2.5-Coder-32B-Instruct	https://huggingface.co/Qwen/-Qwen2.5-Coder-32B-Instruct	32B	Open-source
CodeLlama-34B-Instruct	https://huggingface.co/codellama/CodeLlama-34b-Instruct-hf	34B	
DeepSeek-Coder-33B-Instruct	https://huggingface.co/deepseek-ai/deepseek-coder-33b-instruct	33B	

With IP and AppPatch, we use multiple SOTA LLMs as backends, covering both closed- and open-source models. For closed-source, we choose GPT-4 and Claude-Opus 4.1, two leading models in SE [34]. For open-source, we consider three families, Codellama-x, DeepSeek-Coder-x, and Qwen2.5-Coder-x, all successful in SE [2]. We use their "Instruct", as they are fine-tuned to follow user instructions and hence better adapt to AVR. Notably, we choose open-source LLMs with similar parameter sizes for a fair comparison. Table I summarises all backend LLMs for IP and AppPatch, which we refer to as GPT-4, Claude-O 4.1, Qwen-Coder, CodeLlama, and DS-Coder.

B. Vulnerability Datasets

We leverage the respective vulnerability datasets used in the original evaluations [11], [13] of the selected LLM-based AVR tools. Their authors already processed these datasets to avoid data leakage. Therefore, we directly use them to collect AVR predictions. These datasets are as follows.

- **ExtractFix** [14] and **ZeroDay** [11], [12]: ExtractFix is a widely-used C/C++ vulnerability benchmark for vulnerabil-

TABLE II: **RQ1–VFXplainBench**. The trustworthiness benchmark of LLM-based AVR. IP and APPATCH are evaluated on ExtractFix [14] and ZeroDay [11], [12]. With five different backend LLMs, each tool produces $319 \times 5 = 1,595$ predictions, meaning $1,595 \times 2 = 3,190$ predictions are made on ExtractFix and ZeroDay. Vul-R2 is evaluated on PrimeVul's test set [16] and SVEN [15], with each vulnerability iteratively fixed 10 times. This results in $435 \times 10 = 4,350$ predictions on PrimeVul's test set and $367 \times 10 = 3,670$ predictions on SVEN.

Data	Type	Vulnerabilities	Predictions	Correct Patches	Untrustworthy
ExtractFix [14], ZeroDay [11]	single-function, multi-function, single-hunk, multi-hunk	319	3,190	700	80 (11.4%)
PrimeVul's test set [16]	single-function, single-hunk, multi-hunk	435	4,350	532	98 (18.4%)
SVEN [15]	single-function, single-hunk, multi-hunk	367	3,670	446	111 (24.9%)
Total		1,121	11,210	1,678	289 (17.2%)

ity detection and repair. ZeroDay is a dataset curated by Nong et al. that covers C/C++ vulnerabilities reported after 04/2024. In total, these datasets contain 319 vulnerabilities. They contain both single-hunk vulnerabilities and multi-hunk ones spanning across multiple locations in a single or multiple functions. Details are shown in Appendix B-2. We use these datasets to evaluate IP and APPATCH.

- **PrimeVul** (test set) [16] and **SVEN** [15]: PrimeVul is a large-scale real-world C/C++ dataset. It aggregates vulnerability-related commits from multiple established datasets and applies a rigorous cleaning process. It also adopts a temporally-aware data splitting strategy, resulting in training, validation, and test sets. SVEN manually curates vulnerabilities from multiple C/C++ repositories. Manual inspection [16] demonstrated the label accuracy of 94%. While PrimeVul's training and validation sets are used to tune Vul-R2, its test set and SVEN are used for evaluation, which contain 435 and 367 vulnerabilities, respectively. Both datasets only contain single-hunk vulnerabilities and those that are single-function multi-hunk.

Overall, these datasets contain 1,121 vulnerabilities, covering 57 CWE-IDs. We use them to collect predictions made by the respective LLM-based AVR tools listed in Section III-A.

C. Collecting Predictions

We used the AVR tools in Section III-A to fix the vulnerabilities listed in Section III-B, thus collecting the generated explanations and patches.

Regarding IP and APPATCH, we used five different LLM backends, including both closed- and open-source, as shown in Table I. GPT-4 and Claude-O 4.1 were accessed remotely using OpenAI and Anthropic APIs, respectively. The open-source LLMs were deployed locally via HuggingFace APIs. We used the same configurations, such as temperature, as in the original studies[3]. We replicated their studies on a Rocky Linux server with 24 cores of Intel(R) Xeon(R) Gold 6150 CPU @ 2.70GHz, 256GB RAM, and NVIDIA A100 GPU with 40GB memory. For each setting of using a backend LLM, we applied IP and APPATCH to repair 319 vulnerabilities in ExtractFix and ZeroDay. As a result, we collected 1,595 predictions (319 vulnerabilities $\times$ 5 LLMs) made by IP, and 1,595 predictions made by APPATCH under the same settings.

Regarding Vul-R2, although the authors have not released the model, they provided the generated explanations and patches[4]. Vul-R2 attempts to repair each vulnerability in 10 iterations. This resulted in 4,350 predictions on PrimeVul's test set (435 vulnerabilities $\times$ 10 iterations) and 3,670 predictions on SVEN under the same setting. In conclusion, we collected 11,210 predictions, as shown in Table II.

D. Trustworthiness Labeling

In this paper, we aim to determine if correct patches are generated by LLMs in a trustworthy manner. Therefore, we first identified correct patch predictions. As the evaluated vulnerability datasets mainly lack test cases or exploitation scripts, we follow the original studies [11], [13], manually identifying correct patches. Two authors, each with over five years of experience in program repair, independently assessed the correctness of the generated patches. For each prediction, they were provided with the developer-created patch. They then judged whether the predicted patch is correct, considering both exact matches and program semantic equivalence. In cases of disagreement, a third author, also with over five years of relevant experience, participated in the discussion to reach a consensus. As a result, 1,678 predictions with correct patches were identified, as shown in Table II.

We continued to manually assess the trustworthiness of the correct patch predictions. A correct patch prediction is considered trustworthy if it is justified by a correct explanation. The annotators inspected each explanation, given the developer-created patch, the vulnerability description and the message of the fixing commit. As these records are human-validated, they are reliable reference. The annotators judged whether the prediction is trustworthy, indicated their confidence on a five-point Likert scale, and explicitly documented their rationale. Disagreements were again resolved through discussion with the third annotator. Consequently, among 1,678 correct patch predictions, 289 of them (17.2%) are deemed untrustworthy, as shown in Table II.

Appendix D shows a screenshot of information provided to annotators for labeling. The annotation process achieves the Cohen's Kappa value [35] for inter-reviewer agreement of 0.84, indicating substantial consistency and reliability. These results have significantly bolstered our confidence in both

[3]https://zenodo.org/records/14741018

[4]https://github.com/Xin-Cheng-Wen/Vul-R2

TABLE III: **RQ1–VFXplainBench**. The untrustworthiness between IP that uses zero-shot CoT prompting and APPATCH that applies few-shot CoT prompting. Each combination of AVR tools (IP, APPATCH) and LLM backends (GPT-4, Claude-O 4.1, Qwen-Coder, CodeLlama, DS-Coder) produces 319 predictions

Backend LLM	Predictions	IP (zero-shot)		APPATCH (few-shot)	
		Correct Patches	Untrustworthy	Correct Patches	Untrustworthy
GPT-4	319 per model	91	12 (13.2%)	92 ↑1	6 ↓6 (6.5%)
Claude-O 4.1		129	7 (5.4%)	121 ↓8	9 ↑2 (7.4%)
Qwen-Coder		63	5 (7.9%)	76 ↑13	8 ↑3 (10.5%)
CodeLlama		19	13 (68.4%)	26 ↑7	4 ↓9 (15.4%)
DS-Coder		41	8 (19.5%)	42 ↑1	8 (19.1%)
Total	1,595	343	45 (13.1%)	357 ↑14	35↓10 (9.8%)

Correct Patches: the *higher* the better, Untrustworthy: the *lower* the better
Green and red arrows: improvement and deterioration, respectively

TABLE IV: **RQ1–VFXplainBench**. Comparing the untrustworthiness across the top 25 most dangerous CWE-IDs [36]

CWE-ID	Correct Patches	Untrustworthy
CWE-125	273	51 (18.7%)
CWE-476	255	19 (7.5%)
CWE-787	206	51 (24.8%)
CWE-416	146	62 (42.5%)

CWE-ID	Correct Patches	Untrustworthy
CWE-122	19	0 (0.0%)
CWE-20	14	1 (7.2%)
CWE-200	13	0 (0.0%)
Others	8	3 (37.5%)

VFXplainBench's quality and the evaluation of the detection approach, SUSVF.

E. RQ1: Benchmark Analysis

1) Untrustworthy Predictions: Table II shows that a substantial proportion of correct patch predictions (17.2%) are actually untrustworthy. This means that nearly one in five correct patches generated by LLMs are not trustworthy. The untrustworthiness rate also varies across different datasets. On ExtractFix and ZeroDay, 80 out of 700 predictions (11.4%) are untrustworthy. On PrimeVul's test set, 98 out of 532 predictions (18.4%) are untrustworthy. On SVEN, 111 out of 446 predictions (24.9%) are untrustworthy, showing the worst trustworthiness rate. This variation suggests that trustworthiness may depend on specific characteristics of vulnerabilities.

2) Vulnerability Types: We further compare the trustworthiness of LLM-based AVR tools across different CWE-IDs. Table IV shows that they make no untrustworthy predictions for CWE-122 (Heap-based Buffer Overflow) and CWE-200 (Exposure of Sensitive Information). Predictions for CWE-476 (NULL Pointer Dereference), CWE-20 (Improper Input Validation), CWE-125 (Out-of-bounds Read), and CWE-787 (Out-of-bounds Write) are considerably untrustworthy (7.5%–24.8%). The studied tools especially make a large portion of untrustworthy predictions for CWE-416 (Use After Free) and others (37.5%–42.5%). This finding further emphasises that trustworthiness depends on specific characteristics of vulnerabilities. In Appendix B-2 we discuss the untrustworthiness across single- and multi-hunk vulnerabilities. We find that untrustworthy predictions are more common in single-hunk vulnerabilities than in multi-hunk ones.

Takeaway #1: Even when LLM-based AVR tools produce correct patches, a considerable amount of their predictions (17.2%) are actually untrustworthy. The untrustworthiness varies across different datasets (11.4%–24.9%) and vulnerability types (0%–42.5%).

3) Zero-shot vs. Few-shot: IP leverages zero-shot CoT prompting, while APPATCH applies few-shot CoT prompting. IP and APPATCH use the same set of LLM backends and are evaluated on the same datasets. This allows us to compare their trustworthiness, as shown in Table III. APPATCH, by incorporating relevant contexts and multiple exemplars for few-shot CoT prompting, generates more correct and trustworthy predictions than IP. It produces 14 additional correct patches while reducing untrustworthy predictions by 10, a 25.0% improvement in overall untrustworthiness rate. Across LLM backends, APPATCH all generate more correct patches than IP, except when using Claude-O 4.1. In addition, APPATCH with GPT-4 and CodeLlama make more trustworthy predictions than IP's counterparts. Surprisingly, APPATCH with Claude-O 4.1 and Qwen-Coder is less trustworthy, while DS-Coder remains unchanged.

We further compare two AVR tools using the Scott-Knott Effect Size Difference (ESD) test [37], which leverages hierarchical clustering to partition their means into statistically distinct groups. Results reveal that while the difference in the number of correct patches is not statistically significant, APPATCH statistically significantly decreases the number of untrustworthy predictions.

Takeaway #2: APPATCH, by incorporating relevant contexts and few-shot CoT prompting, is statistically significantly more trustworthy than zero-shot CoT prompting.

4) Evaluation over LLMs: Table III also compares the trustworthiness between LLM backends. Closed-source ones produce the most correct patches, especially Claude-O 4.1 with 129 by IP and 121 by APPATCH. Open-source LLMs generate fewer correct patches, with CodeLlama showing the worst repair success rate. Their trustworthiness also exhibits a similar trend. GPT-4 and Claude-O 4.1 have the lowest untrustworthiness rates (6.5% and 7.4%). Qwen-Coder, CodeLlama, and DS-Coder show higher untrustworthiness rates (10.5%, 15.4%, and 19.1%). Overall, the closed-source LLMs are more effective and trustworthy in AVR than open-source models. We also conduct the Scott-Knott ESD test, which further confirms that these differences are statistically significant.

Takeaway #3: Untrustworthiness occurs in all LLM backends, including the most advanced ones, such as GPT-4 and Claude-O 4.1. The commercial, closed-source LLMs are more effective and trustworthy in AVR than the open-source models.

5) Qualitative Analysis: We analysed 1,678 correct patch predictions in VFXplainBench. We first performed initial open coding [38] on a random set of 75 predictions for preliminary manual analysis. As mentioned in Section III-D, two annotators manually reviewed each prediction's explanation, judging if it correctly justifies the prediction. Disagreement

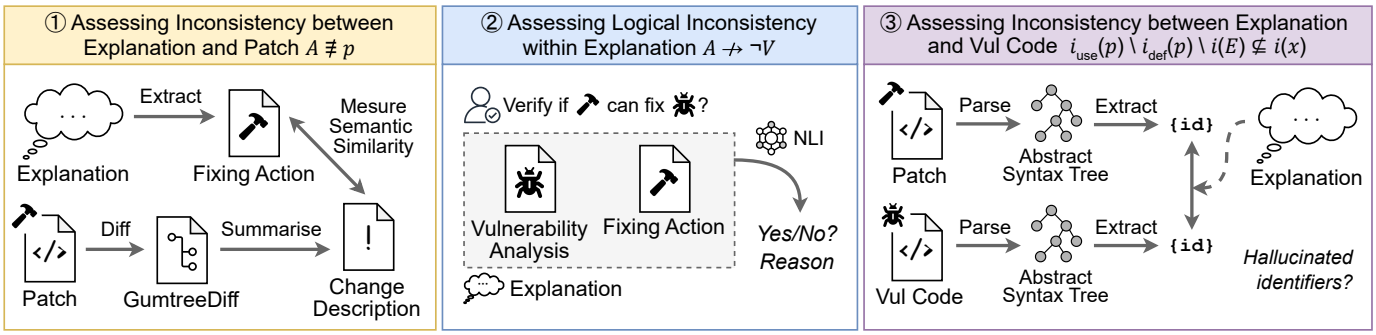


Fig. 2: SUSVF's process consists of three stages. Stage ① assesses the inconsistency between the explanation and the patch, based on the semantic equivalence between the explanation and the change description. Stage ② assesses the explanation's logical inconsistency, particularly between the vulnerability analysis and fixing action. Stage ③ assesses the inconsistency between the explanation and the vulnerable code.

was addressed by involving the third annotator. They all documented their rationale and identified issues by referring to the developer-created patch, the vulnerability description, and the message of the fixing commit. A prediction may be untrustworthy due to multiple issues. Consequently, we summarised their rationales of untrustworthiness and categorised them into three types: Inconsistency between Explanation and Patch, Logical Inconsistency within Explanation, and Inconsistency between Explanation and Vulnerable Code. Then, the remaining predictions were independently annotated by a different group of annotators following the same process. Each annotator identified issues, assessed whether they fit within the three types, and documented the rationale. If any previously unknown issues were identified, we discussed whether they could be accommodated by the existing types or required a new one. Ultimately, we identified three main types based on inconsistencies, along with a few rare cases, as follows.

1) **Inconsistency between Explanation and Patch** (52%). One of the most evident indicators of untrustworthy predictions is a mismatch between the patch and its explanation. In such cases, the proposed fixing action in the explanation does not align with the generated patch. Predictions that exhibit this inconsistency are deemed untrustworthy.
2) **Logical Inconsistency within Explanation** (67%). Another strong indicator of untrustworthy predictions is when the explanation itself is logically inconsistent. Annotators often assess the connection between two main reasoning steps within the explanation, vulnerability analysis and the patching strategy. Intuitively, vulnerability analysis tries to identify the trigger, i.e., the condition under which the vulnerability occurs, while the patching strategy proposes an action to eliminate the vulnerability. However, some explanations show no logical connection between these reasoning steps. In some cases, the LLM even self-admitted and explicitly verbalised its errors or limitations.
3) **Inconsistency between Explanation and Vulnerable Code** (16%). Hallucinations of identifiers, such as variables or APIs [4], are also observed in AVR. These issues lead LLMs to generate explanations and patches that are inconsistent with the vulnerable code. In such cases, LLMs hallucinate by generating patches that use "unknown" identifiers without any prior introduction in the explanation.

We also observed a few cases of untrustworthiness caused by unnatural code modifications or awkward explanations, which do not fall into any of the given types. However, we leave these cases for future work to explore.

Takeaway #4: Inconsistencies can serve as a sufficient condition of untrustworthiness: Inconsistency between Explanation and Patch (52%), Inconsistency within Explanation (67%), and Inconsistency between Explanation and Vulnerable Code (16%).

We are motivated by the results from this section, which show that inconsistencies can serve as a sufficient condition of untrustworthiness. In Section IV, we present an approach that detects untrustworthy predictions by identifying such inconsistencies.

IV. SUSVF: SPOTTING UNTRUSTWORTHY LLM-BASED VULNERABILITY FIXES

We formalise LLM-based AVR tool $M{:}x \mapsto (E, p)$, where x is the vulnerable code, and p is the generated patch. $E = (V, A, \cdot)$ is the explanation that contains an analysis V of the vulnerability and the fixing action A, both in NL. Applying the patch to the vulnerable code obtains the fixed code, $p(x)$. Based on findings in Section III-E5, we consider a prediction $x \mapsto (E, p)$ untrustworthy if it satisfies Equation 1.

$$(A \not\equiv p) \vee (A \not\rightarrow \neg V) \vee \Big(i_{\text{use}}(p) \setminus i_{\text{def}}(p) \setminus i(E) \not\subseteq i(x)\Big) \quad (1)$$

This constraint comprises three components that capture inconsistencies across explanation–patch, logic within the explanation, and explanation–vulnerable code, respectively. Each provides concrete evidence of untrustworthiness. The first component, $A \not\equiv p$, represents cases where the explanation, particularly the fixing action A, is inconsistent with the generated patch p. The second component, $A \not\rightarrow \neg V$, captures hallucinated reasoning steps with conflicting information. If the fixing action A cannot eliminate the vulnerability analysed in V, the prediction is untrustworthy. The last component, $i_{\text{use}}(p) \setminus i_{\text{def}}(p) \setminus i(E) \not\subseteq i(x)$, describes the inconsistency between the explanation and the vulnerable code in terms of

identifiers $i(\cdot)$. The patch p may use identifiers $i_{\text{use}}(p)$ that are not defined $i_{\text{def}}(p)$ or introduced in the explanation $i(E)$ or exist in the vulnerable code $i(x)$.

To this end, we propose SUSVF, an automated approach that can detect untrustworthy predictions based on such inconsistencies. Figure 2 illustrates SUSVF's three-stage process. SUSVF takes as input the vulnerable code, the explanation, and the generated patch. Then, each stage in SUSVF verifies the corresponding inconsistency. *A prediction is considered untrustworthy if it satisfies any constraints representing the inconsistencies*, as defined in Equation 1.

A. Assessing Inconsistency between Explanation and Patch

Intuitively, a prediction should be untrustworthy if the explanation, specifically the fixing action A, fails to justify the generated patch p, i.e., A incorrectly describes p. To identify such cases, we assess the inconsistency between the explanation and the patch by measuring their semantic equivalence. However, this is non-trivial, as the patch p is written in a programming language, whereas the explanation E is in NL. To address this, SUSVF transforms p into an NL description, $p \mapsto d_p \mid p \equiv d_p$. Notably, this stage only focuses on A in E and estimates its semantic equivalence with d_p.

Change Description Generation. This step generates an NL change description to derive the same type of representation for comparison with the fixing action in the explanation. Our key idea is to transform this task into a summarization task, i.e., summarizing code changes in the patch into an NL description. The raw patch operates purely as text and is coarse-grained, as it records line-level diffs. Although it can be used directly, doing so has several limitations. In particular, some changes occur at the token level, such as identifier renaming or operator replacement. As a result, the raw patch can be less explicit in highlighting these changes.

To address this, we extract fine-grained differences between the vulnerable and patched code using specialised source-code differencing tools. Specifically, we leverage Gumtree [17] that parses these two snippets of source code and finds fine-grained changes between them based on their abstract syntax trees (ASTs). This allows us to explicitly capture small changes more directly at the AST node level. Gumtree fine-grains code changes into four types of operations: add, remove, update and move on AST nodes. We then leverage a natural language inference (NLI) machine, such as LLMs, due to their capabilities, to summarise Gumtree's output into an NL summary describing the code changes, denoted by d_p. Gumtree's fine-grained output at the AST node level helps the NLI machine focus on the changes, compared to the raw patch describing changes at the line level.

For example, Figure 1 shows a patch generated by DS-Coder for fixing the the vulnerability CVE-2018-16421. Figure 3 illustrates the process of generating an NL change description. SUSVF first uses Gumtree to find fine-grained differences between the vulnerable and patched code, which is only to add a new subcondition `*out_len < cipher_len-2` into the if condition. SUSVF then feeds Gumtree's output into an NLI machine, e.g., GPT-4o-mini, and instructs it to summarise the changes into a description as "*Adding a subcondition to ensure that *out_len is not less than cipher_len-2*." Compared to direct summarisation without using Gumtree, this explanation is more precise and pinpoints the exact change used to fix the vulnerability.

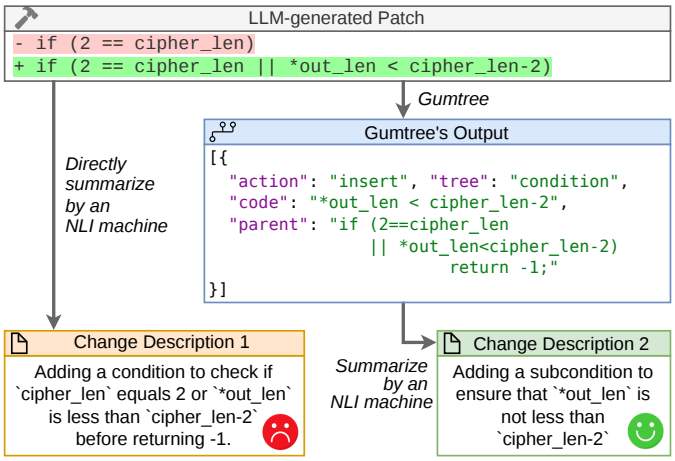


Fig. 3: Describing the generated patch in natural language by utilising an LLM to summarise the fine-grained code changes identified by Gumtree [17].

Semantic Similarity Comparison. To assess inconsistency between the patch and explanation, we compare the generated NL change description d_p and the fixing action A within the explanation based on their semantic equivalence. We leverage several metrics, including Rouge [18], cosine similarity, and LLM-as-a-Judge, following prior work [4].

1) Rouge [18] score is traditionally used in NLP to measure the similarity between a machine-translated summary and reference summaries using overlapping n-grams. In our case, we use it to measure similarity between A and d_p. Following Ullah et al. [4], we use the threshold of 0.34 and mark A and d_p as similar if their Rouge score exceeds this threshold.
2) Cosine Similarity is a metric commonly used in NLP to measure how similar two documents are. We first convert A and d_p into fixed-length vectors using OpenAI's embedding model "*text-embedding-3-small*" and calculate the cosine similarity between them. Similar to Rouge, we use the optimal threshold of 0.84 identified by Ullah et al. and consider A and d_p similar if their cosine similarity exceeds this value.
3) An LLM is used as a judge to evaluate if A and d_p align. If it responds "yes", we assign a score of 1; otherwise, we assign 0.

These metrics may be biased themselves, potentially affecting SUSVF. To mitigate this issue, we apply majority voting [39], a simple yet effective voting method. Specifically, each metric votes on whether $A \not\equiv p$, and the outcome supported by the most metrics wins.

B. Assessing Logical Inconsistency within Explanation

The explanation of a trustworthy prediction must be logically consistent. To this end, SUSVF evaluates the logic

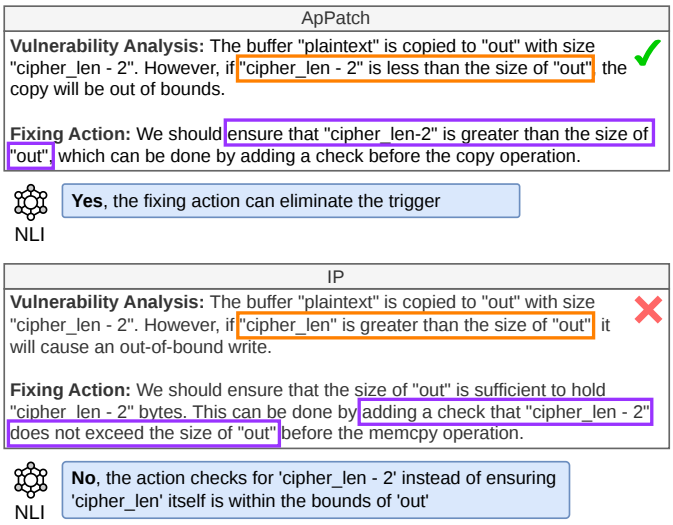


Fig. 4: Evaluate the logical inconsistency within the explanations provided by APPATCH and IP for fixing CVE-2018-16421. The fixing action (purple), the vulnerability trigger (orange), and their consistency verification are identified by an NLI machine, e.g., GPT-4o-mini.

inconsistency among reasoning steps within the explanation, particularly the vulnerability analysis V and the fixing action A, to ensure there are no conflicting reasoning steps. Intuitively, the fixing action A needs to eliminate the vulnerability trigger analysed in V; by applying the fixing action A, the vulnerability trigger is no longer satisfied. If the explanation violates this constraint, $A \not\rightarrow \neg V$ as described in Section III-E5, it is logically inconsistent.

For example, the explanation in Figure 1 include the vulnerability trigger "*if cipher_len is less than 2, plaintext will be out of bounds*" and the fixing action "*ensure that cipher_len is greater than 2, which can be done by adding a check before the copy operation*". According to the explanation, the vulnerability is triggered when cipher_len is less than 2, and adding a check to ensure that cipher_len is greater than 2 can eliminate this trigger. Hence, this explanation seems logically consistent. Note that this prediction is still untrustworthy due to the inconsistency between the patch and the explanation.

The vulnerability trigger V and the fixing action A are explicitly stated in NL within the explanation E. Given the specific trigger V and action A, we assess whether A can eliminate V by framing this problem as an NLI task. Unlike AVR, which requires substantial domain knowledge, this task inherently assesses the logical consistency between two NL statements. We leverage LLMs' NLI capabilities to address this task. An LLM is instructed to follow four outlined steps with few-shot learning. We provide four examples of LLM-provided explanations, two logically consistent and two logically inconsistent, which are sufficient for effective few-shot learning [40]. Based on the manual annotation described in Section III, we select four representative explanations and craft the analysis accordingly. The prompt template can be found in Appendix 1.

Figure 4 is an illustration of this stage. It shows two explanations provided by APPATCH and IP for fixing CVE-2018-16421. The fixing action A and the vulnerability trigger V are highlighted in purple and orange, respectively. By instructing the LLM to evaluate whether A can eliminate V, it is revealed that the DS-APPATCH's explanation is consistent, while IP's is not, as its action only checks 'cipher_len–2' instead of ensuring 'cipher_len' is within the bounds of 'out'.

C. Assessing Inconsistency between Explanation and Code

This stage evaluates the explanation with respect to the vulnerable code. Based on the common issues found in Section III-E5, we compare identifiers between the vulnerable code x and the patch p, with the explanation E as the reference. The vulnerable code and the explanation are deemed inconsistent, $i_{\text{use}}(p) \setminus i_{\text{def}}(p) \setminus i(E) \not\subseteq i(x)$, if the patch uses hallucinated identifiers that do not exist in the explanation or the vulnerable code. We employ a lightweight approach to detect hallucinated identifiers. In particular, SUSVF first parses the vulnerable code and the patched code into abstract syntax trees (ASTs) using the parser available in Gumtree. We then compare these ASTs with the explanation as a supporting reference to identify hallucinated identifiers.

The LLM may hallucinate and use "unknown" variables or APIs in the patch that are not defined or introduced in the explanation, raising trustworthiness concerns. To detect this, we compare identifiers defined and used in the vulnerable code and the patch, using the explanation as a reference. We first extract all identifiers in the vulnerable code and the patched code by traversing the ASTs and applying rule-matching based on the type of the current AST node and its parent. SUSVF then finds "unknown" identifiers that are absent from the vulnerable code by comparing two sets of identifiers. For each "unknown" identifiers, we traverse the patched code's AST to verify if there exists a corresponding declaration node. A final search is conducted in the explanation to filter out identifiers introduced or defined in the explanation without literal code. This procedure results in a list of used identifiers without definitions in the patch. A non-empty list indicates that the LLM is likely to hallucinate when making the AVR prediction.

V. EVALUATION OF SUSVF

This section describes a series of experiments conducted to evaluate how well SUSVF detects untrustworthy predictions made by LLM-based AVR systems.

A. Experimental Setup

In addition to RQ1 in Section III, we also study the following RQs:

RQ2 [Effectiveness]. *How effective is* SUSVF *at identifying untrustworthy predictions among correct patch predictions?* We evaluate SUSVF's effectiveness on VFXplainBench constructed in Section III. We frame this task as a binary classification problem with trustworthy and untrustworthy as negative and positive labels, respectively. We then compare the effectiveness of SUSVF against LLM-as-a-Judge baselines.

RQ3 [Efficiency]. *How efficient is* SUSVF *in identifying untrustworthy predictions?* We focus on the cost-efficiency of

SUSVF by analyzing both processing time (in seconds) and the monetary cost (in USD) of using LLM APIs per prediction.

RQ4 [Ablation]. *How different design choices impact* SUSVF*?* We first compare SUSVF with an alternative approach that summarises code changes directly from raw patches, without relying on GumTree. Next, we examine the impact of using a single metric to assess semantic equivalence between the patch and its explanation, rather than an ensemble of metrics. Finally, we analyse how each stage of SUSVF influences its overall effectiveness and efficiency.

RQ5 [Sensitivity]. *How different NLI machines affect* SUSVF*?* We observe how SUSVF changes when using different LLMs as the NLI machine. For LLM-as-a-Judge baselines, we employ strong commercial models available at the time of writing, namely GPT-4 and Claude-Opus 4.1, to achieve the best performance. In contrast, for SusVF's NLI machine, due to resource constraints, we intentionally use lightweight and cost-efficient models from the same families, including GPT-3.5-turbo, GPT-4o-mini, and Claude-Haiku-4.1. For open-source LLMs for NLI, we select the strongest-performing family in VFXplainBench, Qwen2.5-Coder, as shown in Table III, and evaluate multiple model scales (1.5B, 7B, 14B, and 32B) to examine the impact of model size.

Metrics: To assess effectiveness, we use standard binary classification metrics, including *Precision*, *Recall* (TPR), *Specificity* (TNR), and *Matthews Correlation Coefficient* (MCC). Precision measures the proportion of truly untrustworthy predictions among all detected ones. Recall measures the proportion of correctly detected untrustworthy predictions among all truly untrustworthy ones. Specificity measures the proportion of correctly detected trustworthy predictions among all truly trustworthy predictions. MCC is a balanced metric for evaluating the quality of binary classifiers, 1 indicates perfect classification, 0 corresponds to random classification, and –1 reflects inverse classification.

To assess efficiency, we report the processing time (in seconds) and the cost of using APIs (in USD) per prediction.

Baseline: We compare SUSVF against LLM-as-a-Judge [19]. All five backend models shown in Table I are used as judges, with each method denoted by the prefix Judge–. We access GPT-4 and Claude-O 4.1 remotely using OpenAI and Anthropic APIs, respectively, while the open-source LLMs are deployed locally via HuggingFace APIs. The prompt template is presented in Appendix 2.

Dataset: We leverage VFXplainBench described in Table II to evaluate the ability of detecting untrustworthy predictions. VFXplainBench contains 1,678 correct patch predictions for fixing vulnerabilities. These vulnerabilities cover different types, including single-function, multi-function, single-hunk, and multi-hunk [11], [13]. Keep in mind that SUSVF is not inherently limited to single-function, single-hunk vulnerabilities. This is because Gumtree, the used static analysis tool, can parse a whole repository and find code differences at the repository level, while NLI is code-agnostic.

Implementation: We use GPT-4o-mini as SUSVF's default NLI machine, then analyse the impact of different LLMs in RQ5. We set all LLMs' temperature to 0. Experiments are conducted on a Rocky Linux server, 24 cores of Intel(R) Xeon(R) Gold 6150 CPU @ 2.70GHz, 256GB RAM, NVIDIA A100 GPU, 40GB memory.

TABLE V: **RQ2**. The effectiveness of SUSVF and the LLM-as-a-Judge baselines on VFXplainBench

Method	Pre	Rec	Spe	MCC
SusVF (ours)	0.882	0.732	0.980	0.768
Judge-GPT-4	0.251‡	0.404‡	0.751‡	0.131‡
Judge-Claude-O-4.1	0.356‡	0.544‡	0.797‡	0.292‡
Judge-Qwen-Coder	0.401‡	0.261‡	0.919‡	0.216‡
Judge-CodeLlama	0.238‡	0.139‡	0.908‡	0.059‡
Judge-DS-Coder	0.128‡	0.017‡	0.976‡	-0.018‡

and : the best and second best values in each column, respectively
Two-sided t-test †: p-value < 0.05, ‡: p-value < 0.01

B. RQ2: Effectiveness

Compared with Baselines. Table V shows that the LLM-as-a-Judge baselines perform poorly. While they show high Specificity of 0.751–0.976, their Precision (0.128–0.401) and Recall (0.017–0.544) are relatively low. This means that they often fail to understand the nuances of untrustworthiness and misclassify AVR predictions as trustworthy. Their low MCC (-0.018–0.292) further indicate a negligible to weak correlation between the baselines' judgements and actual untrustworthiness. This suggests that LLM-as-a-Judge approaches provide unreliable assessments of prediction trustworthiness. In addition, we find no meaningful differences between self-validation, i.e., using the same LLM to evaluate its own predictions, and cross-validation, that is, evaluating a LLM's predictions by a different LLM.

In contrast, SUSVF consistently demonstrates strong effectiveness, with Precision of 0.882, Recall of 0.732, and Specificity of 0.980. Its MCC of 0.768 also shows a strong correlation between SUSVF's judgements and untrustworthiness. Compared to the baselines, SUSVF improves by up to 589% in Precision, 4,206% in Recall, 31% in Specificity, and 4,367% in MCC. All improvements are statistically significant, confirming SUSVF's superior effectiveness and generalisability across AVR predictions.

Top 25 Most Dangerous CWE-IDs. We report SUSVF's effectiveness across the top 25 most dangerous CWE-IDs [36] in Appendix B-1. SUSVF maintains generally strong effectiveness, with Precision of 0.808–1, Recall of 0.733–1, and Specificity of 0.887–1. However, its effectiveness is more moderate for certain vulnerabilities (e.g., CWE-125). This indicates that some types are more challenging for detecting untrustworthy predictions.

Single-Hunk vs. Multi-Hunk. Appendix B-2 reports SUSVF's effectiveness across single- and multi-hunk vulnerabilities. Overall, SUSVF performs consistently well in both cases, with the effectiveness on single-hunk vulnerabilities slightly higher by 7% in MCC.

Effectiveness Across AVR Systems. The effectiveness of SUSVF on different LLM-based AVR systems is also presented in Appendix B-3. We evaluated 11 system settings: Vul-R2 and 10 combinations of five LLM backends with two approaches (IP and ApPatch). For most settings, SUSVF

TABLE VI: **RQ3–RQ4**. The efficiency of SUSVF on VFX-plainBench. We report the average processing time (in seconds) and API's cost (in $) per prediction

(a) Compare w/ baselines

Method	Time	API
SusVF (ours)	1.78	0.001
Judge-GPT-4	4.73	0.044
Judge-Claude-O-4.1	12.57	0.059
Judge-Qwen-Coder	31.62	-
Judge-CodeLlama	24.80	-
Judge-DS-Coder	19.05	-

(b) Ablation study

Method	Time	API
w/o Gumtree	1.71	0.001
w/ only Rouge	1.73	0.001
w/ only Cosine Sim.	1.70	0.001
w/ only An LLM	1.63	0.001
w/o ① Expl–Patch	1.29	0.001
w/o ② Expl's Logic	1.46	0.001
w/o ③ Expl–Code	1.75	0.001

achieves Precision of 0.750–1, Recall of 0.625–0.846, Specificity of 0.966–1, and MCC of 0.713–0.852. For IP w/ Qwen-Coder and APPATCH w/ CodeLlama (only 4–5 untrustworthy examples), results are relatively lower, with Precision of 0.6, Recall of 0.5–0.6, and MCC of 0.549–0.677.

Case Study. We report and analyze a few cases of false positives (FPs) and false negatives (FNs) in Appendix C. Overall, FPs mainly occur because Stage ① cannot capture the semantic equivalence between the patch and the explanation in challenging cases. Another reason is that NLI in Stage ② may fail when vulnerability conditions and fixing actions are too complex. FNs are mostly due to NLI hallucinations. Our current implementation uses a lightweight, cost-efficient LLM (i.e., GPT-4o-mini) as the NLI machine. We believe stronger LLMs can reduce FPs and FNs.

Takeaway #5: LLM-as-a-Judge baselines fail to detect untrustworthy AVR predictions. In contrast, SUSVF consistently and significantly outperforms the baselines across all metrics. It demonstrates a strong correlation with untrustworthiness and maintains good effectiveness across AVR predictions and systems.

C. RQ3: Efficiency

Table VIa reports SUSVF's efficiency in handling a prediction, measured by average processing time (in seconds) and the cost of using the APIs (in $). On average, per prediction, SUSVF requires 1.78 seconds and $0.001 due to the use of OpenAI's APIs for the NLI machine. Compared to the LLM-as-a-Judge methods, SUSVF achieves the best balance of time and $. All baselines are slower than SUSVF, requiring 4.73–31.62 seconds per prediction. Commercial LLMs, Judge-GPT-4 and Judge-Claude-O-4.1, are also 44–59 times more expensive ($0.044–$0.059). In contrast, open-source LLMs, Qwen-Coder, CodeLlama, and DS-Coder, deployed locally as judges, incur no API cost but are much slower (19.05–31.62 seconds). While the monetary cost of invoking these LLMs' APIs is 0, there are indirect costs, e.g., electricity and GPU, which we do not count in this study. As the remote LLMs run on more powerful machines than our local deployment for the open-source LLMs, as described in Section V-A, it is expected that these local LLMs are less time-efficient.

TABLE VII: **RQ4**. Comparing SUSVF with its ablated versions: an approach directly using raw patches without relying on Gumtree, three approaches that use a single metric to assess patch–explanation inconsistency, and three approaches that exclude each stage in the three-stage process of SUSVF

Method	Pre	Rec	Spe	MCC
SusVF (ours)	0.882	0.732	0.980	0.768
w/o Gumtree	$0.598^{\dagger}$	$0.659^{\ddagger}$	$0.909^{\dagger}$	$0.546^{\ddagger}$
w/ only Rouge	$0.539^{\ddagger}$	$0.822^{\dagger}$	$0.855^{\ddagger}$	$0.580^{\ddagger}$
w/ only Cosine Similarity	$0.262^{\ddagger}$	$0.948^{\dagger}$	$0.449^{\ddagger}$	$0.309^{\ddagger}$
w/ only An LLM	$0.552^{\ddagger}$	$0.815^{\dagger}$	$0.863^{\ddagger}$	$0.588^{\dagger}$
w/o ① Expl–Patch	$1.000^{\ddagger}$	$0.551^{\ddagger}$	$1.000^{\ddagger}$	0.709
w/o ② Expl's Logic	$0.750^{\ddagger}$	$0.293^{\ddagger}$	$0.960^{\ddagger}$	$0.411^{\ddagger}$
w/o ③ Expl–Code	$0.879^{\ddagger}$	0.707	$0.960^{\ddagger}$	$0.750^{\ddagger}$

: the best value in each column, : the worst value in each column
Two-sided t-test †: p-value < 0.05, ‡: p-value < 0.01

Takeaway #6: SUSVF requires 1.78 seconds and $0.001 for using GPT-4o-mini as the NLI machine per prediction. It is substantially more money-efficient than commercial LLMs-as-a-Judge, while also being far faster than locally deployed ones.

D. RQ4: Ablation Study

Effectiveness: We first compare the effectiveness of SUSVF with a variant that summarises LLM-generated patches without using Gumtree (w/o Gumtree). As shown in Table VII, excluding Gumtree overall reduces the effectiveness, with statistically significant drops by 32% in Precision, 10% in Recall, 7% in Specificity, and 29% in MCC. The results show that leveraging Gumtree can significantly improve SUSVF's detection of untrustworthiness.

We also examine the impact of using a single metric to assess semantic equivalence between the patch and its explanation in Stage ①, rather than an ensemble of metrics. As shown in Table VII, although all three variants of SUSVF achieve higher Recall (by 11%–30%), their Precision, Specificity, and MCC decrease significantly. In particular, the variant using only cosine similarity performs the worst, with reductions of 70% in Precision, 54% in Specificity, and 60% in MCC. These results indicate that a single metric is insufficient for reliably assessing semantic equivalence, leading to a bias toward incorrectly classifying predictions as untrustworthy.

Table VII also compares SUSVF's effectiveness with its ablated versions, each omitting one stage of the three-stage process: assessing the inconsistency between the explanation and the patch, assessing the explanation's logical inconsistency, or assessing the inconsistency between the explanation and the vulnerable code. Removing Stage ① statistically significantly drops Recall by 25%. MCC also drops by 8%; however, this difference is not statistically significant. Both Precision and Specificity reach 1, indicating that SUSVF is now biased to label predictions as trustworthy. Removing Stage ② leads to statistically significant decreases in all metrics. Particularly, it drops Precision by 15%, Recall by 60%, Specificity by 2%, and MCC by 47%. Removing Stage ③ also decreases SUSVF's effectiveness by 1% in Precision, 3% in Recall,

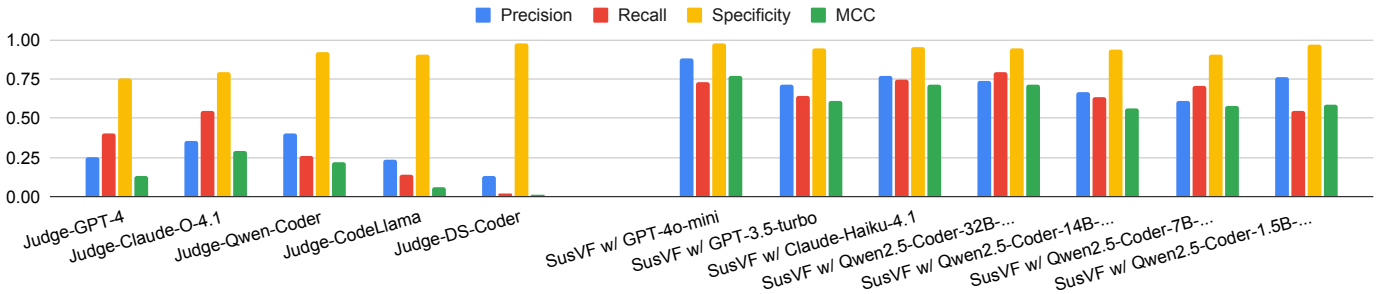


Fig. 5: **RQ5**. The effectiveness of LLM-as-a-Judge and SusVF (ours) with different LLMs as the NLI machine.

2% in Specificity, and 2% in MCC. These differences are statistically significant, except for Recall.

> **Takeaway #7:** Gumtree, the ensemble of metrics in Stage ①, and all three stages play important roles in SusVF's effectiveness. Excluding them leads to statistically significant decreases by up to 60% in MCC.

Efficiency: Table VIb shows that using Gumtree, the ensemble in Stage ①, and the last two stages have no big impact on SusVF's efficiency. The process time and the cost of using OpenAI's APIs per AVR prediction remain nearly unchanged. These components appear to be computationally lightweight. In contrast, Stage ① adds 0.49 seconds (28%) to processing time, indicating that most overhead comes from comparing change descriptions with explanations.

> **Takeaway #8:** Most of SusVF's overhead comes from comparing change descriptions with explanations (e.g., 28% of the total processing time). The other components in SusVF have a negligible impact on efficiency.

E. RQ5: Sensitivity Study

We investigate how the choice of LLMs used as the NLI machine influences SusVF's effectiveness. The results are shown in Figure 5. Details are presented in Appendix B-4.

Compared with LLM-as-a-Judge baselines, SusVF statistically significantly outperforms them across all NLI backends. Specifically, it achieves 52%–589% higher Precision, 0%–4,553% higher Recall, and 93%–4,367% higher MCC. Improvements in Specificity are more modest (up to 30%). Notably, SusVF with Qwen2.5-Coder-32B-Instruct also outperforms its corresponding Judge baseline.

Among the NLI machines, GPT-4o-mini, Claude-Haiku-4.1, and Qwen2.5-Coder-32B-Instruct achieve the highest effectiveness, with Precision of 0.737–0.882, Recall of 0.732–0.791, Specificity of 0.942–0.980, and MCC of 0.711–0.768. The other LLMs show relatively lower effectiveness, with Precision of 0.611–0.765, Recall of 0.544–0.707, Specificity of 0.907–0.966, and MCC of 0.581–0.610. While Qwen2.5-Coder-32B-Instruct demonstrates the best effectiveness among the Qwen2.5-Coder-x-Instruct LLMs, we observe no big differences between the three smaller respective LLMs.

> **Takeaway #9:** SusVF with GPT-4o-mini, Claude-Haiku-4.1, and Qwen2.5-Coder-32B-Instruct achieve the best effectiveness. SusVF consistently outperforms LLM-as-a-Judge across all NLI machines, especially when the NLI machine and the LLM-as-a-Judge are the same model.

VI. Discussion

Implications for AVR. Trustworthiness in LLM-based AVR poses a fundamental challenge that extends beyond patch correctness. As the adoption of LLMs accelerates, our findings demonstrate that patch correctness alone is insufficient for ensuring reliability. This is because a considerable proportion of correct patches can be generated with inconsistencies in their explanations. This aligns with prior work, for example, Khati et al. [9] conducted a survey with developers and found that interpretability of generated outputs is clearly important for building trust. Together, these findings suggest the need for a paradigm shift in how LLM-based AVR systems are designed, trained, and evaluated, with a stronger emphasis on human–LLM alignment. This need was further emphasised by Zhou et al. [2]. They highlighted the importance of supporting integration into developer workflows and enabling effective interaction and collaboration between developers and LLM-based systems. Current AVR benchmarks mainly prioritise functional correctness, assuming that correct patches reflect sound reasoning. Our results invalidate this assumption and suggest that future benchmarks should explicitly capture trustworthiness. Benchmarks that are trustworthiness-oriented would enable a more in-depth evaluation of LLMs and discourage surface-level correctness.

Benefits of Detecting Untrustworthy Predictions. During the *development* of LLM-based AVR systems, automated detection of untrustworthy predictions plays a central role in assessing and improving system quality and reliability. It facilitates the review of AVR outputs, enabling automated trustworthiness testing to be integrated into continuous integration and delivery pipelines. This supports the iterative development process, where both performance and trustworthiness must be continuously evaluated and refined [28], [29]. In addition, untrustworthy predictions provide valuable diagnostic signals that help identify deeper behavioural issues in LLM reasoning and guide the improvements of LLM-based AVR systems.

During *deployment*, detection methods serve as a safeguard within operational pipelines by flagging untrustworthy predictions and reducing the risk of misleading or unsafe

repairs. This is particularly important in security-sensitive environments, where incorrect vulnerability fixes may have severe consequences. They also enable real-time monitoring of LLM-based AVR systems in DevOps and online testing workflows, allowing verification of prediction trustworthiness under real-world and corner-case inputs. Hence, these methods foster greater human trust in AVR systems.

Limitations and Future Work. SUSVF encodes three inconsistency criteria derived from our analysis of VFXplainBench. Nevertheless, there are a few cases that do not fit into these criteria. This is because inconsistency is a *sufficient condition* for untrustworthiness. If the explanation is inconsistent, the prediction is untrustworthy, but not vice versa. Appendix C presents a few FPs and FNs, and analyzes their potential causes. Future work can address these cases to further improve the effectiveness of SUSVF.

In addition, learning-based assessments in Stage ① and Stage ② could be inaccurate. For example, the use of rouge and cosine similarity may be limited in some cases, while LLMs can hallucinate. We can strengthen SUSVF by incorporating advanced semantic measures and formal methods. For instance, vulnerability triggers and fixing actions could be encoded as logical formulas and analysed using Satisfiability Modulo Theories (SMT) solvers to assess their consistency. Exploring such techniques represents a promising direction for enhancing SUSVF in the future.

VII. Threats to Validity

Construct Validity. Several potential threats are worth discussing. VFXplainBench relies on publicly available AVR resources and approaches that were carefully curated and validated in previous research [2], [10]–[13]. We further manually analysed their predictions, achieving a Cohen's Kappa of 0.84 for inter-reviewer agreement. These results suggest substantial consistency and reliability. We acknowledge that the labeling is imperfect, leaving a small portion of mislabeled data. However, we believe the results will still hold good.

Internal Validity. A threat can be related to the bias of the three metrics, rouge, cosine similarity, and LLM, used to assess patch–explanation consistency. To minimize this threat, we apply majority voting to limit false positives. A further threat lies in possible inaccuracies of the tools and LLMs used in SUSVF, e.g., Gumtree and the NLI machine. We addressed this by following best practices [40] and validated tools in prior studies [17]. However, we acknowledge that some bias may remain in some cases. In this respect, future empirical investigations may provide further insights.

External Validity. One key threat is VFXplainBench's diversity. While it was constructed from existing vulnerability datasets and AVR systems validated in previous research [11], [13], they may not fully capture the diversity of vulnerabilities. While this remains a limitation of our work, please note that VFXplainBench includes predictions made by multiple AVR systems using various LLM backends over 57 vulnerability types across various projects. This provides a degree of diversity that supports the generalizability of our findings. Still, broader studies covering more models and vulnerabilities may reveal further insights into the trustworthiness of LLM-based AVR. In addition, our study focuses exclusively on C/C++ programs. While C/C++ is widely common in vulnerability analysis, its vulnerabilities may differ from those in other languages. Therefore, our findings may not be directly applicable to other languages. Future work should evaluate the trustworthiness of LLM-based AVR across more languages to test whether our insights generalise.

Conclusion Validity. We interpreted the results using multiple well-established metrics: Precision, Recall, Specificity, MCC. We also used appropriate statistical tests, the Scott-Knott Effect Size Difference test and the two-sided t-test, to support our findings.

VIII. Related Work

LLM-based AVR. Prior studies [2] have identified two major categories of LLM-based AVR systems: (1) fine-tuning, which updates LLM parameters with labelled datasets, and (2) prompt engineering, which keeps parameters fixed while designing prompts to guide responses.

Regarding fine-tuning, studies have incorporated additional inputs beyond the vulnerable code to enhance AVR, such as ASTs, vulnerability descriptions, CWE examples, and related commits [2]. Other studies [30], [41] have also modified LLM architectures for better AVR. For example, Wen et al. [13] fine-tuned a reasoning LLM called Vul-R2 for AVR. They combined a reasoning learning module and a curriculum-based verifiable rewarded training module to generate both patches and explanations.

Regarding prompt engineering, researchers have explored *zero-shot prompting* to repair vulnerabilities without training. Prompts often include the vulnerable code [42], [43], the vulnerability locations [13], and/or vulnerability-related semantic details [44]. Other studies have also applied *few-shot prompting* by providing a few repair examples [45] or *CoT prompting* to instruct LLMs to follow several intermediate reasoning steps [12], [32], [46]. Various approaches, such as APPATCH [11], have been introduced to automatically retrieve relevant contexts to assist prompting.

Despite these advances, current paradigms mainly focus on generating correct patches without examining LLMs' decision-making process. Hence, it remains unclear whether LLMs interpret and repair vulnerabilities in a trustworthy manner.

Trustworthiness of LLMs in SE. LLMs have been rapidly applied in both industry and academia for various tasks. As they become increasingly integral to critical tasks, ensuring their trustworthiness is essential. Prior studies [47] have conceptualised trustworthiness across multiple dimensions, such as safety, fairness, explainability, ethics, robustness, and privacy. Following such concepts, researchers [6] have raised concerns about LLMs' trustworthiness in various domains and proposed countermeasures.

In SE, studies [9], [23] have also defined trustworthiness and distinguished it from trust. Khati et al. [9] conducted a human study across multiple SE tasks, including program repair, and highlighted interpretability alongside accuracy as the two most critical factors. Unfortunately, existing work

has shown that LLMs often struggle with interpretability in vulnerability analysis. Ullah et al. [4] showed that LLMs cannot reliably identify and reason about vulnerabilities, as they provide incorrect and unfaithful reasoning.

Recent efforts to evaluate and improve LLMs have mostly focused on accuracy and robustness. Li et al. [48] introduced SV-TrustEval-C, a benchmark that evaluates LLMs' abilities of vulnerability analysis under diverse data- and control-flow complexities and structural and semantic perturbations. Their results show that current LLMs are far from satisfactory, relying more on pattern matching than on robust reasoning.

Although situated in the domain of LLM-based AVR trustworthiness, our work is the first systematically evaluate trustworthiness. Existing benchmarks and systems primarily assess *what* predictions LLMs produce (e.g., correctness, robustness) but rarely examine *why* those predictions are made. To bridge this gap, we propose VFXplainBench, a benchmark that evaluates the trustworthiness by leveraging LLM-generated explanations. Furthermore, we introduce SUSVF, the first approach that detects untrustworthy LLM-based AVR predictions.

IX. CONCLUSION

This paper constructed VFXplainBench, a benchmark of 11,210 predictions made by three AVR systems, namely instruction prompting, APPATCH, and Vul-R2. VFXplainBench revealed that a considerable portion is untrustworthy, even when the generated patches are correct. Thus it motivated us to propose SUSVF, an automated approach that detects untrustworthy predictions among correct patch predictions by analysing inconsistencies between patch, explanation, vulnerable code, and reasoning steps. Results on VFXplainBench show that SUSVF significantly outperforms LLM-as-a-Judge baselines, demonstrating its effectiveness and efficiency. These findings underscore the importance of evaluating LLMs not only for their correctness but also for trustworthiness, and highlight the potential of automated explainability-based methods to accelerate auditing and foster safer deployment of LLM-based AVR. Future work can explore rare cases and incorporate formal methods to strengthen SUSVF.

REFERENCES

[1] X. Zhou, K. Kim, B. Xu, D. Han, and D. Lo, "Out of sight, out of mind: Better automatic vulnerability repair by broadening input ranges and sources," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: https://doi.org/10.1145/3597503.3639222

[2] X. Zhou, S. Cao, X. Sun, and D. Lo, "Large language model for vulnerability detection and repair: Literature review and the road ahead," *ACM Trans. Softw. Eng. Methodol.*, Dec. 2024, just Accepted. [Online]. Available: https://doi.org/10.1145/3708522

[3] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language models don't always say what they think: unfaithful explanations in chain-of-thought prompting," in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS '23. Red Hook, NY, USA: Curran Associates Inc., 2023.

[4] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, "Llms cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks," in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024, pp. 862–880.

[5] J. Kong, X. Xie, and S. Liu, "Demystifying memorization in llm-based program repair via a general hypothesis testing framework," *Proc. ACM Softw. Eng.*, vol. 2, no. FSE, Jun. 2025. [Online]. Available: https://doi.org/10.1145/3729390

[6] L. Nguyen Tung, S. Cho, X. Du, N. Neelofar, V. Terragni, S. Ruberto, and A. Aleti, "Automated trustworthiness oracle generation for machine learning text classifiers," *Proc. ACM Softw. Eng.*, vol. 2, no. FSE, Jun. 2025. [Online]. Available: https://doi.org/10.1145/3729376

[7] J. Wei, X. Wang, D. Schuurmans, M. Bosma, b. ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 24 824–24 837. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf

[8] Y. Noller, R. Shariffdeen, X. Gao, and A. Roychoudhury, "Trust enhancement issues in program repair," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 2228–2240. [Online]. Available: https://doi.org/10.1145/3510003.3510040

[9] D. Khati, Y. Liu, D. N. Palacio, Y. Zhang, and D. Poshyvanyk, "Mapping the trust terrain: Llms in software engineering - insights and perspectives," *ACM Trans. Softw. Eng. Methodol.*, Oct. 2025, just Accepted. [Online]. Available: https://doi.org/10.1145/3771282

[10] Y. Li, F. H. Shezan, B. Wei, G. Wang, and Y. Tian, *SoK: towards effective automated vulnerability repair*. USA: USENIX Association, 2025.

[11] Y. Nong, H. Yang, L. Cheng, H. Hu, and H. Cai, *APPATCH: automated adaptive prompting large language models for real-world software vulnerability patching*. USA: USENIX Association, 2025.

[12] Y. Nong, M. Aldeen, L. Cheng, H. Hu, F. Chen, and H. Cai, "Chain-of-thought prompting of large language models for discovering and fixing software vulnerabilities," 2024. [Online]. Available: https://arxiv.org/abs/2402.17230

[13] X.-C. Wen, Z. Lin, Y. Yang, C. Gao, and D. Ye, "Vul-r2: A reasoning llm for automated vulnerability repair," in *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '25, 2025. [Online]. Available: https://arxiv.org/abs/2510.05480

[14] X. Gao, B. Wang, G. J. Duck, R. Ji, Y. Xiong, and A. Roychoudhury, "Beyond tests: Program vulnerability repair via crash constraint extraction," *ACM Trans. Softw. Eng. Methodol.*, vol. 30, no. 2, Feb. 2021. [Online]. Available: https://doi.org/10.1145/3418461

[15] J. He and M. Vechev, "Large language models for code: Security hardening and adversarial testing," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 1865–1879. [Online]. Available: https://doi.org/10.1145/3576915.3623175

[16] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, and Y. Chen, " Vulnerability Detection with Code Language Models: How Far are We? ," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2025, pp. 1729–1741. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/ICSE55347.2025.00038

[17] J.-R. Falleri and M. Martinez, "Fine-grained, accurate and scalable source differencing," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: https://doi.org/10.1145/3597503.3639148

[18] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*. Barcelona, Spain: Association for Computational Linguistics, Jul. 2004, pp. 74–81. [Online]. Available: https://aclanthology.org/W04-1013/

[19] J. He, J. Shi, T. Y. Zhuo, C. Treude, J. Sun, Z. Xing, X. Du, and D. Lo, "From code to courtroom: Llms as the new software judges," 2025. [Online]. Available: https://arxiv.org/abs/2503.02246

[20] J. Long, "Large language model guided tree-of-thought," 2023. [Online]. Available: https://arxiv.org/abs/2305.08291

[21] Y. Yao, Z. Li, and H. Zhao, "Beyond chain-of-thought, effective graph-of-thought reasoning in language models," 2024. [Online]. Available: https://arxiv.org/abs/2305.16582

[22] A. Aleti, B. Ray, R. Hoda, and S. Chen, "Trustworthy ai software engineers," 2026. [Online]. Available: https://arxiv.org/abs/2602.06310

[23] L. Kastner, M. Langer, V. Lazar, A. Schomacker, T. Speith, and S. Sterz, " On the Relation of Trust and Explainability: Why to Engineer

for Trustworthiness ," in *2021 IEEE 29th International Requirements Engineering Conference Workshops (REW)*. Los Alamitos, CA, USA: IEEE Computer Society, Sep. 2021, pp. 169–175. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/REW53955.2021.00031

[24] Z. Fei, J. Ge, C. Li, T. Wang, Y. Li, H. Zhang, L. Huang, and B. Luo, "Patch correctness assessment: A survey," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 2, Jan. 2025. [Online]. Available: https://doi.org/10.1145/3702972

[25] F. Liu, Y. Liu, L. Shi, Z. Yang, L. Zhang, X. Lian, Z. Li, and Y. Ma, " Beyond Functional Correctness: Exploring Hallucinations in LLM-Generated Code ," *IEEE Transactions on Software Engineering*, vol. 52, no. 03, pp. 1037–1055, Mar. 2026. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/TSE.2026.3657432

[26] M. Böhme, E. O. Soremekun, S. Chattopadhyay, E. Ugherughe, and A. Zeller, "Where is the bug and how is it fixed? an experiment with practitioners," in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, ser. ESEC/FSE 2017. New York, NY, USA: Association for Computing Machinery, 2017, p. 117–128. [Online]. Available: https://doi.org/10.1145/3106237.3106255

[27] E. Winter, D. Bowes, S. Counsell, T. Hall, S. Haraldsson, V. Nowack, and J. Woodward, "How do developers really feel about bug fixing? directions for automatic program repair," *IEEE Transactions on Software Engineering*, vol. 49, no. 4, pp. 1823–1841, 2023.

[28] V. Riccio, G. Jahangirova, A. Stocco, N. Humbatova, M. Weiss, and P. Tonella, "Testing machine learning based systems: a systematic mapping," *Empirical Software Engineering*, vol. 25, no. 6, pp. 5193–5254, Nov 2020. [Online]. Available: https://doi.org/10.1007/s10664-020-09881-0

[29] S. Martínez-Fernández, J. Bogner, X. Franch, M. Oriol, J. Siebert, A. Trendowicz, A. M. Vollmer, and S. Wagner, "Software engineering for ai-based systems: A survey," *ACM Trans. Softw. Eng. Methodol.*, vol. 31, no. 2, Apr. 2022. [Online]. Available: https://doi.org/10.1145/3487043

[30] M. Fu, C. Tantithamthavorn, T. Le, V. Nguyen, and D. Phung, "Vulrepair: a t5-based automated software vulnerability repair," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2022. New York, NY, USA: Association for Computing Machinery, 2022, p. 935–947. [Online]. Available: https://doi.org/10.1145/3540250.3549098

[31] Z. Yu, Z. Guo, Y. Wu, J. Yu, M. Xu, D. Mu, Y. Chen, and X. Xing, *PATCHAGENT: a practical program repair agent mimicking human expertise*. USA: USENIX Association, 2025.

[32] Y. Kim, S. Shin, H. Kim, and J. Yoon, "Logs in, patches out: Automated vulnerability repair via {Tree-of-Thought}{LLM} analysis," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 4401–4419.

[33] D. Votipka, S. Rabin, K. Micinski, J. S. Foster, and M. L. Mazurek, "An observational investigation of reverse Engineers' processes," in *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, Aug. 2020, pp. 1875–1892. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/votipka-observational

[34] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, "SWE-bench: Can language models resolve real-world github issues?" in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=VTF8yNQM66

[35] M. L. McHugh, "Interrater reliability: the kappa statistic," *Biochemia medica*, vol. 22, no. 3, pp. 276–282, 2012.

[36] MITRE Corporation, "2025 cwe top 25 most dangerous software weaknesses," https://cwe.mitre.org/top25/archive/2025/2025_cwe_top25.html, 2025, accessed: 2026-01-26.

[37] C. Tantithamthavorn, S. McIntosh, A. E. Hassan, and K. Matsumoto, "The impact of automated parameter optimization on defect prediction models," *IEEE Transactions on Software Engineering*, vol. 45, no. 7, pp. 683–711, 2019.

[38] S. H. Khandkar, "Open coding," *University of Calgary*, vol. 23, no. 2009, p. 2009, 2009.

[39] F. Leon, S.-A. Floria, and C. Bădică, "Evaluating the effect of voting methods on ensemble-based classification," in *2017 IEEE International Conference on INnovations in Intelligent SysTems and Applications (INISTA)*, 2017, pp. 1–6.

[40] U. Halil, J. Huang, D. Graux, and J. Z. Pan, "Llm shots: Best fired at system or user prompts?" in *Companion Proceedings of the ACM on Web Conference 2025*, ser. WWW '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1605–1613. [Online]. Available: https://doi.org/10.1145/3701716.3717814

[41] M. Fu, V. Nguyen, C. Tantithamthavorn, D. Phung, and T. Le, "Vision transformer inspired automated vulnerability repair," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 3, Mar. 2024. [Online]. Available: https://doi.org/10.1145/3632746

[42] H. Pearce, B. Tan, B. Ahmad, R. Karri, and B. Dolan-Gavitt, " Examining Zero-Shot Vulnerability Repair with Large Language Models ," in *2023 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2023, pp. 2339–2356. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/SP46215.2023.10179420

[43] B. Ahmad, S. Thakur, B. Tan, R. Karri, and H. Pearce, "On hardware security bug code fixes by prompting large language models," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 4043–4057, 2024.

[44] Y. Luo, B. Li, A. Singhal, P. Tseng, L. Zhang, Q. Zou, X. Sun, and P. Liu, "Exploring prompt patterns for effective vulnerability repair in real-world code by large language models," in *Proceedings of the 10th ACM International Workshop on Security and Privacy Analytics*, ser. IWSPA '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 23–33. [Online]. Available: https://doi.org/10.1145/3716815.3729010

[45] M. Fu, C. K. Tantithamthavorn, V. Nguyen, and T. Le, " ChatGPT for Vulnerability Detection, Classification, and Repair: How Far Are We? ," in *2023 30th Asia-Pacific Software Engineering Conference (APSEC)*. Los Alamitos, CA, USA: IEEE Computer Society, Dec. 2023, pp. 632–636. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/APSEC60848.2023.00085

[46] U. Kulsum, H. Zhu, B. Xu, and M. d'Amorim, "A case study of llm for automated vulnerability repair: Assessing impact of reasoning and patch validation feedback," in *Proceedings of the 1st ACM International Conference on AI-Powered Software*, ser. AIware 2024. New York, NY, USA: Association for Computing Machinery, 2024, p. 103–111. [Online]. Available: https://doi.org/10.1145/3664646.3664770

[47] Y. Liu, Y. Yao, J.-F. Ton, X. Zhang, R. Guo, H. Cheng, Y. Klochkov, M. F. Taufiq, and H. Li, "Trustworthy LLMs: a survey and guideline for evaluating large language models' alignment," in *Socially Responsible Language Modelling Research*, 2023. [Online]. Available: https://openreview.net/forum?id=oss9uaPFfB

[48] Y. Li, P. Branco, A. M. Hoole, M. Marwah, H. M. Koduvely, G.-V. Jourdan, and S. Jou, " SV-TrustEval-C: Evaluating Structure and Semantic Reasoning in Large Language Models for Source Code Vulnerability Analysis ," in *2025 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2025, pp. 3014–3032. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/SP61157.2025.00191